Model Assessment: Predicting density and assessing factors driving marbled murrelet distribution in Puget Sound using hierarchical distance sampling

Code by S.J. Gillman

Required Packages

```
library(tidyverse)
library(MCMCvis)
library(nimble)
library(ggplot2)
```

Load Data

Required Files Previously Made or Provided

- `Survey_Grid_Info.RData`
- `chain_1.rds`
- `chain_2.rds`
- `chain_3.rds`
- `nimConstants.rds`
- `nimData.rds`
- `gof_SummaryStats.rds`
- `N_posterior_gof.rds`
- `rN_posterior_gof.rds`
- `gs_posterior_gof.rds`
- `pcap_posterior_gof1-6.rds`
- `lambda_posterior_gof1-6.rds`

pcap and lambda `.rds` files that end in the number are the iterations, 1 = 1 to 1000, 2 = 1001 to 2000, 3 = 2001 to 3000, etc. b,c,d are the PSU-Segments 2001 to 8095 broken up into 2k groups due to overall large size and needing to load to github.

```
load("../DataAnalysis/DataFiles/Survey_Grid_Info.RData")
samples_1 <- readRDS("RESULTS/COMPRESSED/chain_1.rds")
samples_2 <- readRDS("RESULTS/COMPRESSED/chain_2.rds")
samples_3 <- readRDS("RESULTS/COMPRESSED/chain_3.rds")

# Model input data
nimConstants <- readRDS("RESULTS/COMPRESSED/nimConstants.rds")
nimData <- readRDS("RESULTS/COMPRESSED/nimData.rds")
```

```r
# Summarized GoF INFO
results_gof <- readRDS("RESULTS/GOF/COMPRESSED/gof_SummaryStats.rds")

N_sub <- readRDS("RESULTS/COMPRESSED/GOF/N_posterior_gof.rds")
rN_sub <- readRDS("RESULTS/COMPRESSED/GOF/rN_posterior_gof.rds")

gs_sub <- readRDS("RESULTS/COMPRESSED/GOF/gs_posterior_gof.rds")

lambda_sub <- rbind(
  cbind(
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof1.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof1b.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof1c.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof1d.rds")),
  cbind(
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof2.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof2b.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof2c.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof2d.rds")),
  cbind(
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof3.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof3b.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof3c.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof3d.rds")),
  cbind(
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof4.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof4b.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof4c.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof4d.rds")),
  cbind(
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof5.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof5b.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof5c.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof5d.rds")),
  cbind(
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof6.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof6b.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof6c.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/lambda_posterior_gof6d.rds")))


pcap_sub <- rbind(
  cbind(
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof1.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof1b.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof1c.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof1d.rds")),
  cbind(
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof2.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof2b.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof2c.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof2d.rds")),
  cbind(
```

```r
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof3.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof3b.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof3c.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof3d.rds")),
  cbind(
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof4.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof4b.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof4c.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof4d.rds")),
  cbind(
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof5.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof5b.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof5c.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof5d.rds")),
  cbind(
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof6.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof6b.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof6c.rds"),
    readRDS("RESULTS/COMPRESSED/GOF/pcap_posterior_gof6d.rds")))
```

```r
chains <- coda::mcmc.list(samples_1,samples_2,samples_3)

result_summary <- MCMCsummary(chains, probs = c(0.025,0.25,0.5,0.75, 0.975)) %>%
  rownames_to_column("parameter") %>%
  mutate(
    variable = sub("\\[.*", "", parameter),
    model_id = gsub("\\]", "", sub(".*\\[", "", parameter)))

round(max(result_summary$Rhat, na.rm =T),2)
```

```
## [1] 1
```

```r
pd_dir <- bayestestR::p_direction(chains)

round(max(results_gof$Rhat, na.rm =T),2)
```

```
## [1] 1.17
```

N GoF

```r
# result matrix
N_new <- matrix(0, nrow = nimConstants$nS, ncol = nrow(N_sub))

# Loop over sites
for (r in 1:nimConstants$nS) {
  my_idx <- nimConstants$MONTHYEAR[r]

  # Extract vectors for all iterations at once
  rN_vec <- rN_sub[, my_idx]
  lambda_vec <- lambda_sub[, r]
```

```r
  # Calculate p
  prob_p <- rN_vec / (rN_vec + lambda_vec)

  # Create new N estimates
  N_new[r, ] <- rnbinom(nrow(N_sub), size = rN_vec, prob = prob_p)
}

# Residuals for 'observed'
FT_obs <- colSums((sqrt(t(N_sub)) - sqrt(t(lambda_sub)))^2)
# Residuals for 'new'
FT_sim <- colSums((sqrt(N_new) - sqrt(t(lambda_sub)))^2)

# Posterior Predictive P-value
round(mean(FT_sim > FT_obs),2)
```

```
## [1] 0.88
```

GS GoF

```r
# result matrix
GS_new <- matrix(0, nrow = nimConstants$nS, ncol = nrow(gs_sub))
E_gs <- matrix(0, nrow = nimConstants$nS, ncol = nrow(gs_sub))

for (r in 1:nimConstants$nS) {
  my_idx <- nimConstants$MONTHYEAR[r]

  gs_vec <- gs_sub[, my_idx]
  E_gs[r, ] <- nimData$C[r] * gs_sub[, my_idx]
  GS_new[r, ] <- rpois(nrow(gs_sub), E_gs[r, ])
}

# Residuals for 'observed'
FT_obs <- colSums((sqrt(nimData$GS) - sqrt(E_gs))^2)
# Residuals for 'new'
FT_sim <- colSums((sqrt(GS_new) - sqrt(E_gs))^2)

# Posterior Predictive P-value
mean(FT_sim > FT_obs)
```

```
## [1] 1
```

Counts GoF

```r
C_actual <- nimData$C


C_new <- matrix(0, nrow = nimConstants$nS, ncol = nrow(N_sub))
C_expected <- matrix(0, nrow = nimConstants$nS, ncol = nrow(N_sub))
```

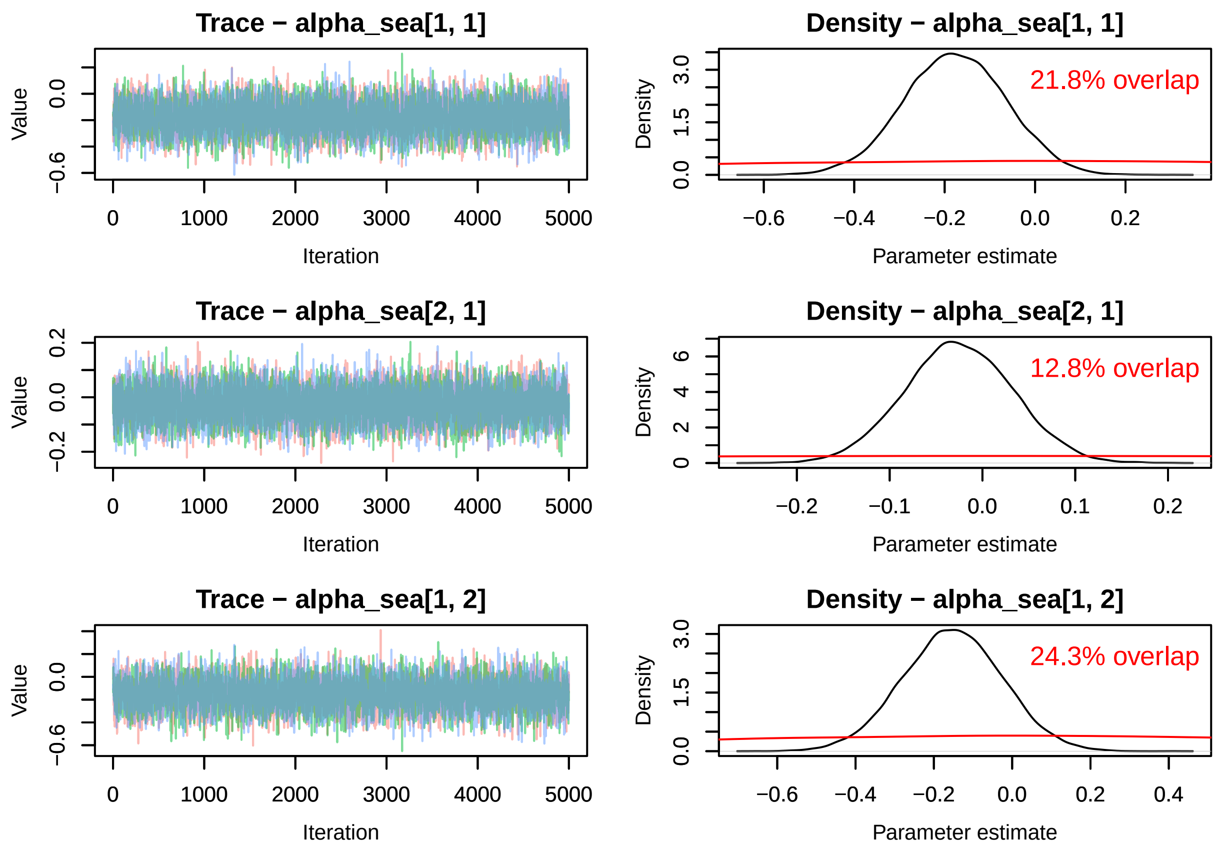
```r
for (r in 1:nimConstants$nS) {
  N_vec    <- N_sub[, r]
  pcap_vec <- pcap_sub[, r]

  # expected Counts
  C_expected[r,] <- N_sub[, r] * pcap_sub[, r]
  # Create new C estimates
  C_new[r, ] <- rbinom(nrow(N_sub), size = N_vec, prob = pcap_vec)
}

# Residuals for 'observed'
FT_obs <- colSums((sqrt(nimData$C) - sqrt(C_expected))^2)
# Residuals for 'new'
FT_sim <- colSums((sqrt(C_new) - sqrt(C_expected))^2)

#Posterior Predictive P-value
round(mean(FT_sim > FT_obs), 2)
```

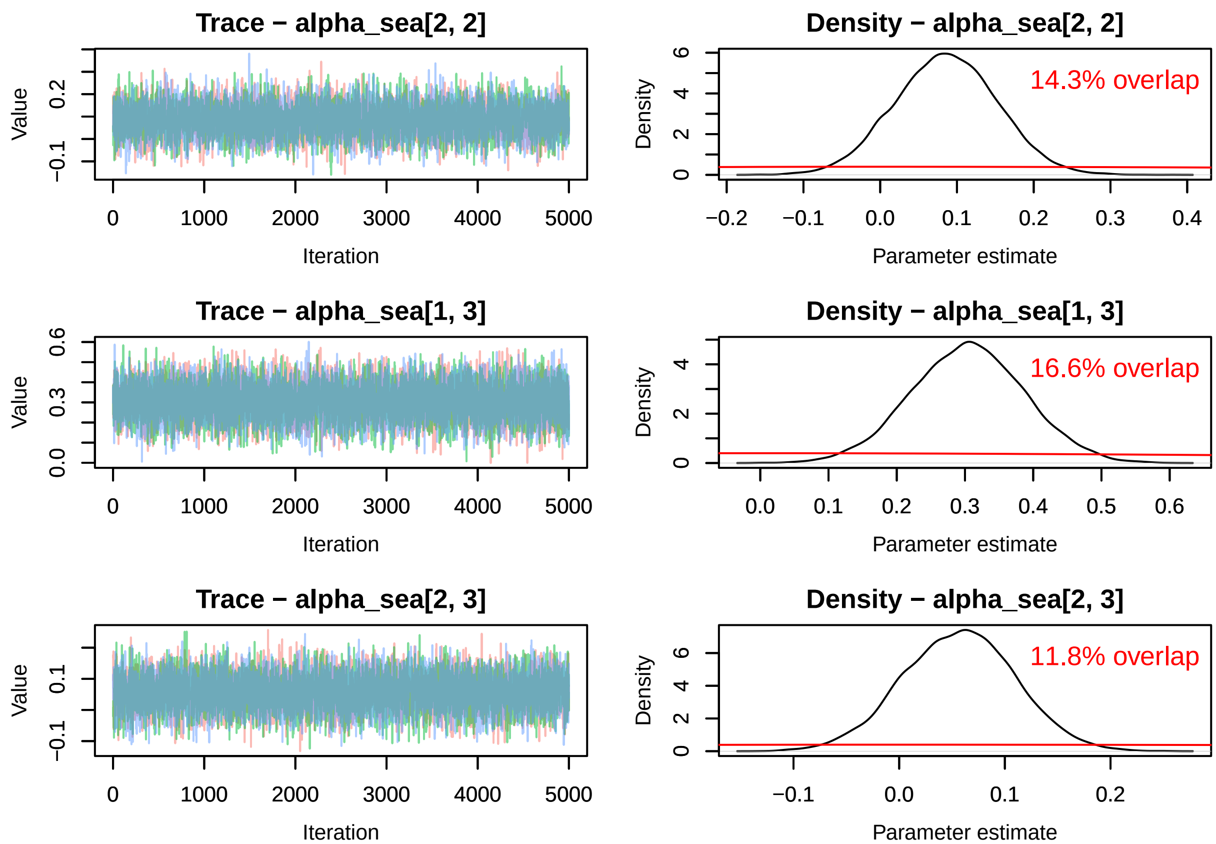
```
## [1] 0.53
```

Prior posterior overlap

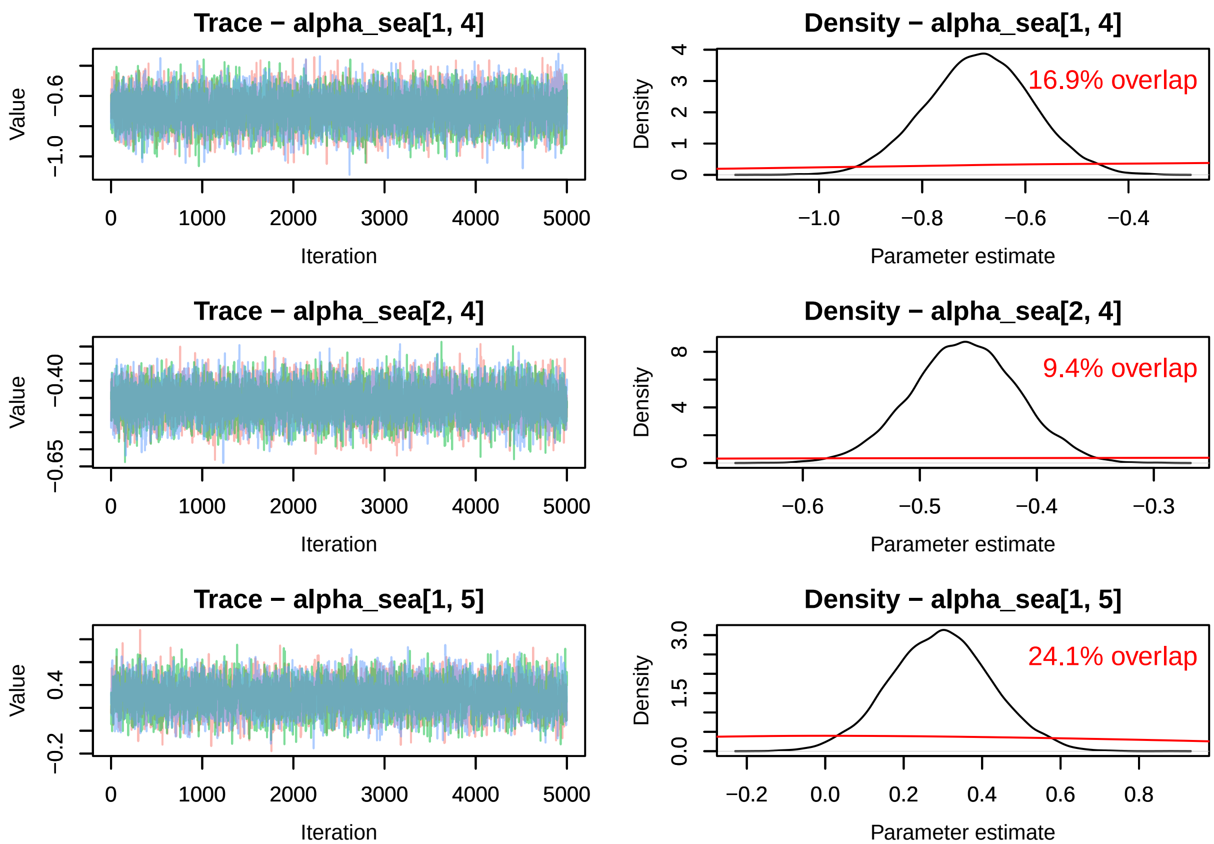
```r
PR <- rnorm(15000, 0, 1)

MCMCtrace(chains, params = 'alpha_sea', priors = PR, pdf = FALSE)
```

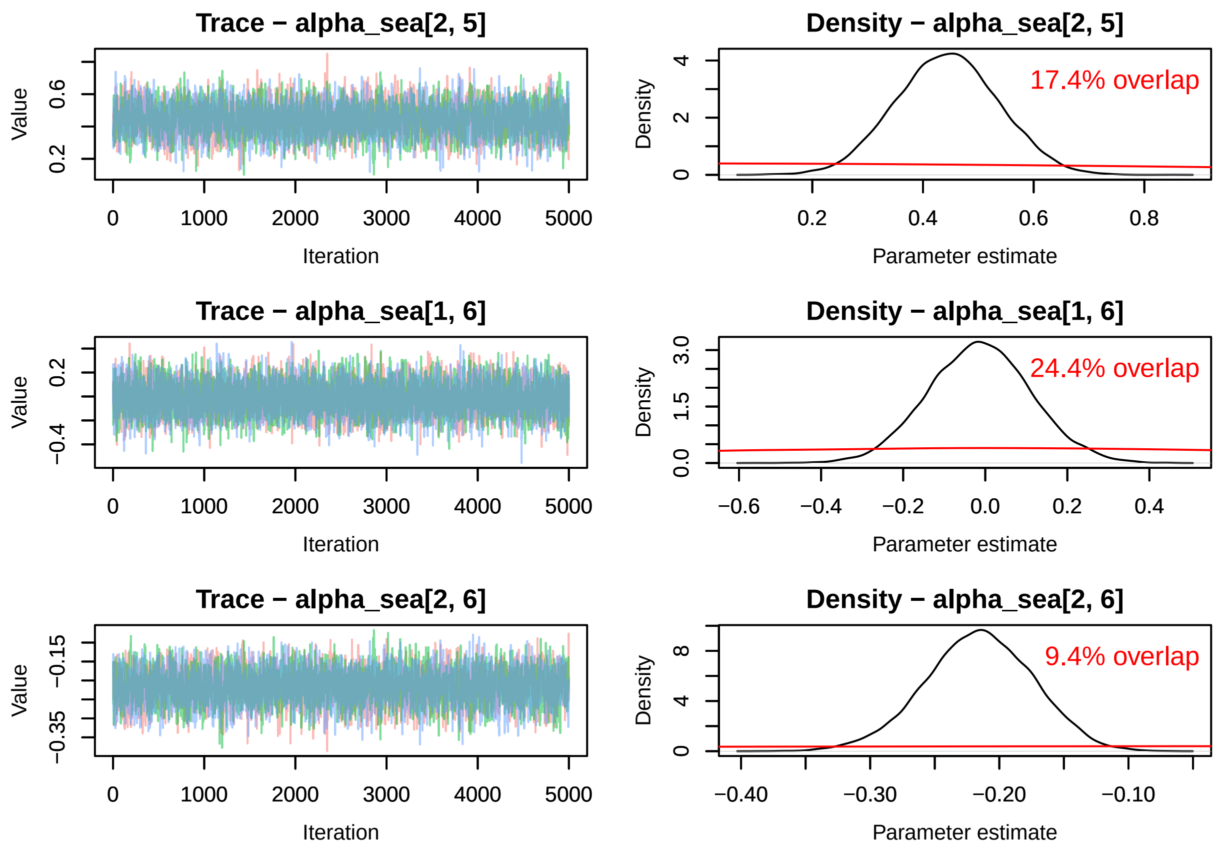
```
## Warning in MCMCtrace(chains, params = "alpha_sea", priors = PR, pdf = FALSE):
## Only one prior specified for > 1 parameter. Using a single prior for all
## parameters.
```


Trace − alpha_sea[1, 1]
Value
Iteration
Density − alpha_sea[1, 1]
Density
Parameter estimate
21.8% overlap
Trace − alpha_sea[2, 1]
Value
Iteration
Density − alpha_sea[2, 1]
Density
Parameter estimate
12.8% overlap
Trace − alpha_sea[1, 2]
Value
Iteration
Density − alpha_sea[1, 2]
Density
Parameter estimate
24.3% overlap



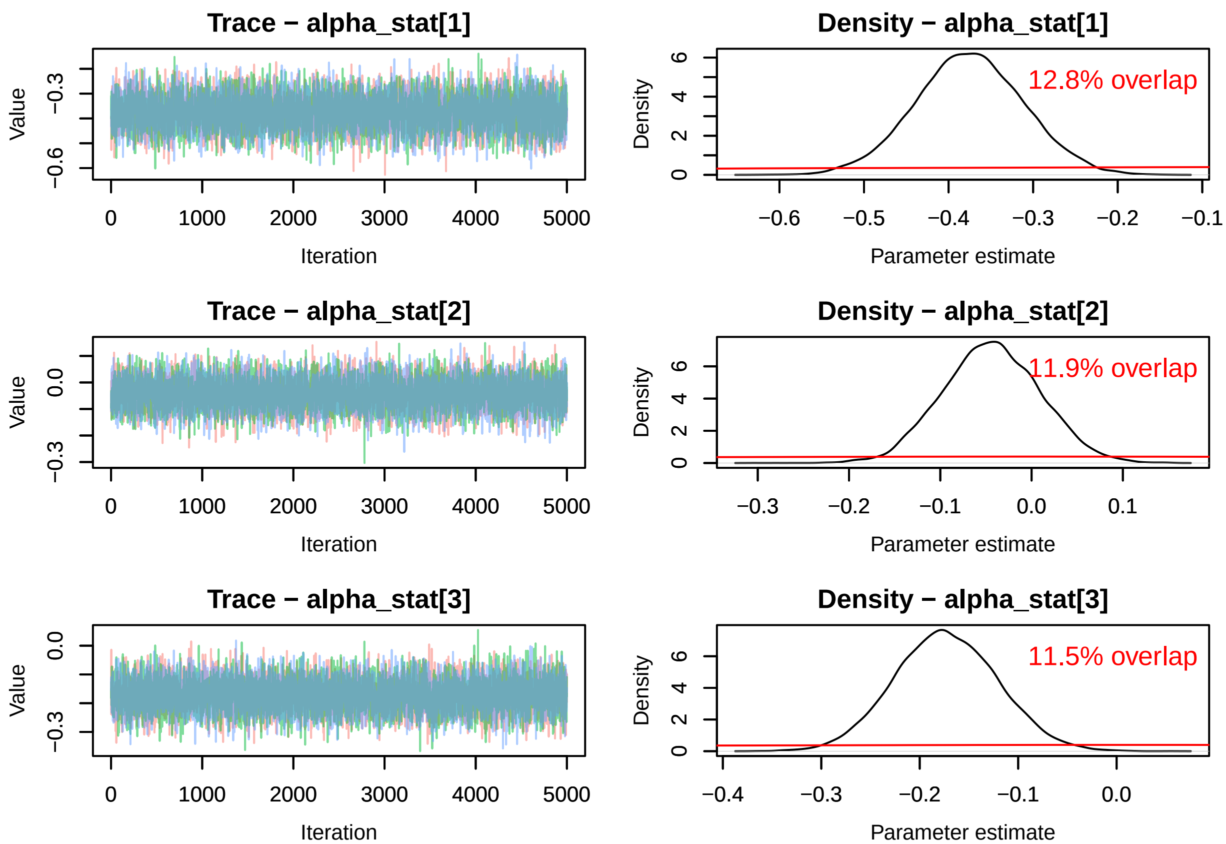
Trace − alpha_sea[2, 2]
Value
Iteration
Density − alpha_sea[2, 2]
14.3% overlap
Density
Parameter estimate
Trace − alpha_sea[1, 3]
Value
Iteration
Density − alpha_sea[1, 3]
16.6% overlap
Density
Parameter estimate
Trace − alpha_sea[2, 3]
Value
Iteration
Density − alpha_sea[2, 3]
11.8% overlap
Density
Parameter estimate

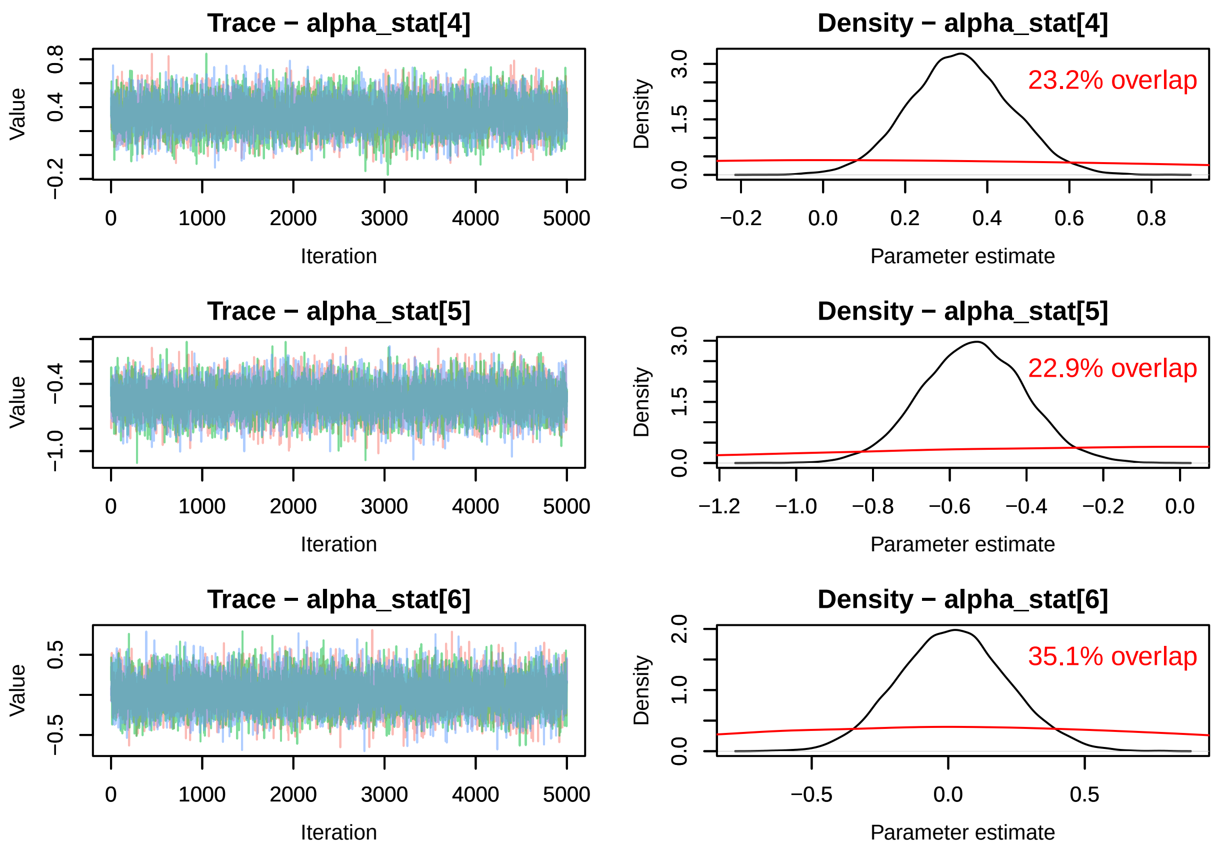

Trace − alpha_sea[1, 4]
Value
−1.0
−0.6
0
1000
2000
3000
4000
5000
Iteration
Density − alpha_sea[1, 4]
Density
0
1
2
3
4
16.9% overlap
−1.0
−0.8
−0.6
−0.4
Parameter estimate
Trace − alpha_sea[2, 4]
Value
−0.65
−0.40
0
1000
2000
3000
4000
5000
Iteration
Density − alpha_sea[2, 4]
Density
0
4
8
9.4% overlap
−0.6
−0.5
−0.4
−0.3
Parameter estimate
Trace − alpha_sea[1, 5]
Value
−0.2
0.4
0
1000
2000
3000
4000
5000
Iteration
Density − alpha_sea[1, 5]
Density
0.0
1.5
3.0
24.1% overlap
−0.2
0.0
0.2
0.4
0.6
0.8
Parameter estimate

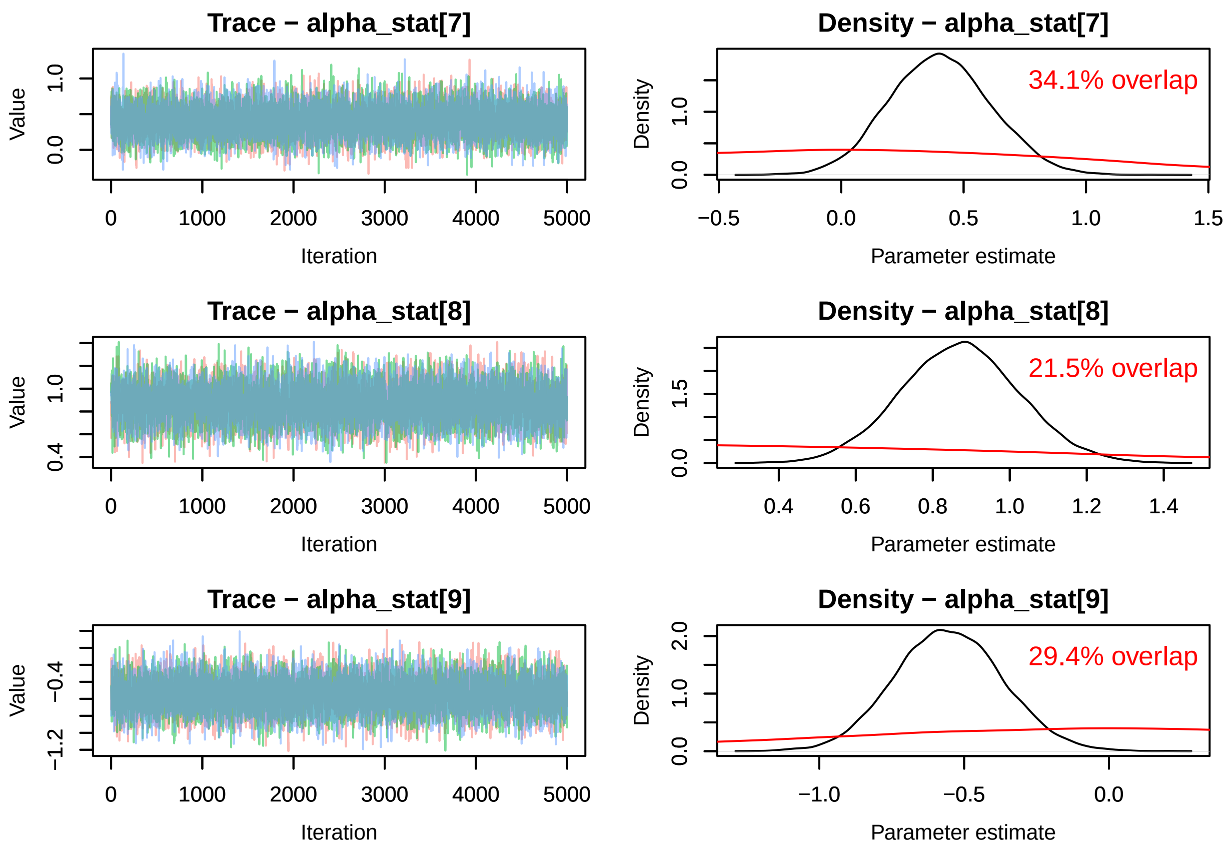


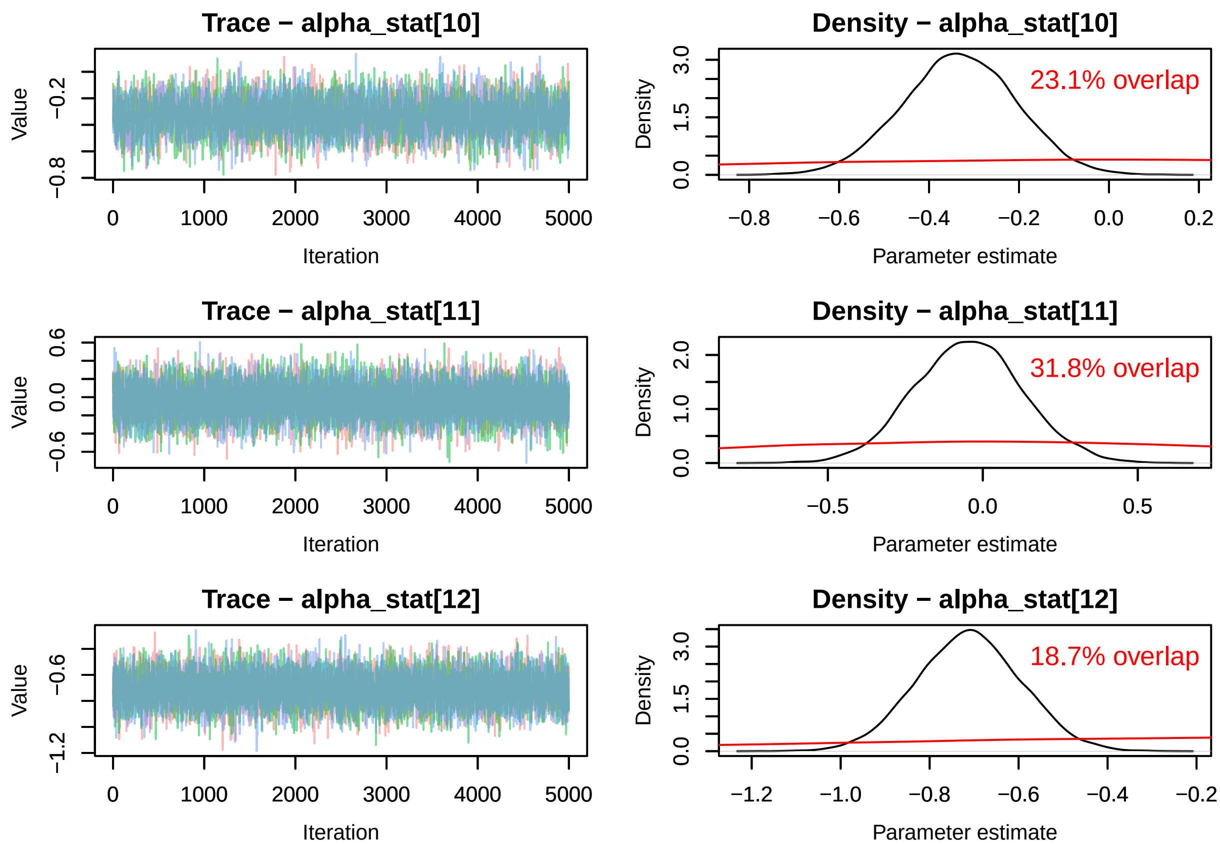

```r
MCMCtrace(chains, params = 'alpha_stat', priors = PR, pdf = FALSE)
```

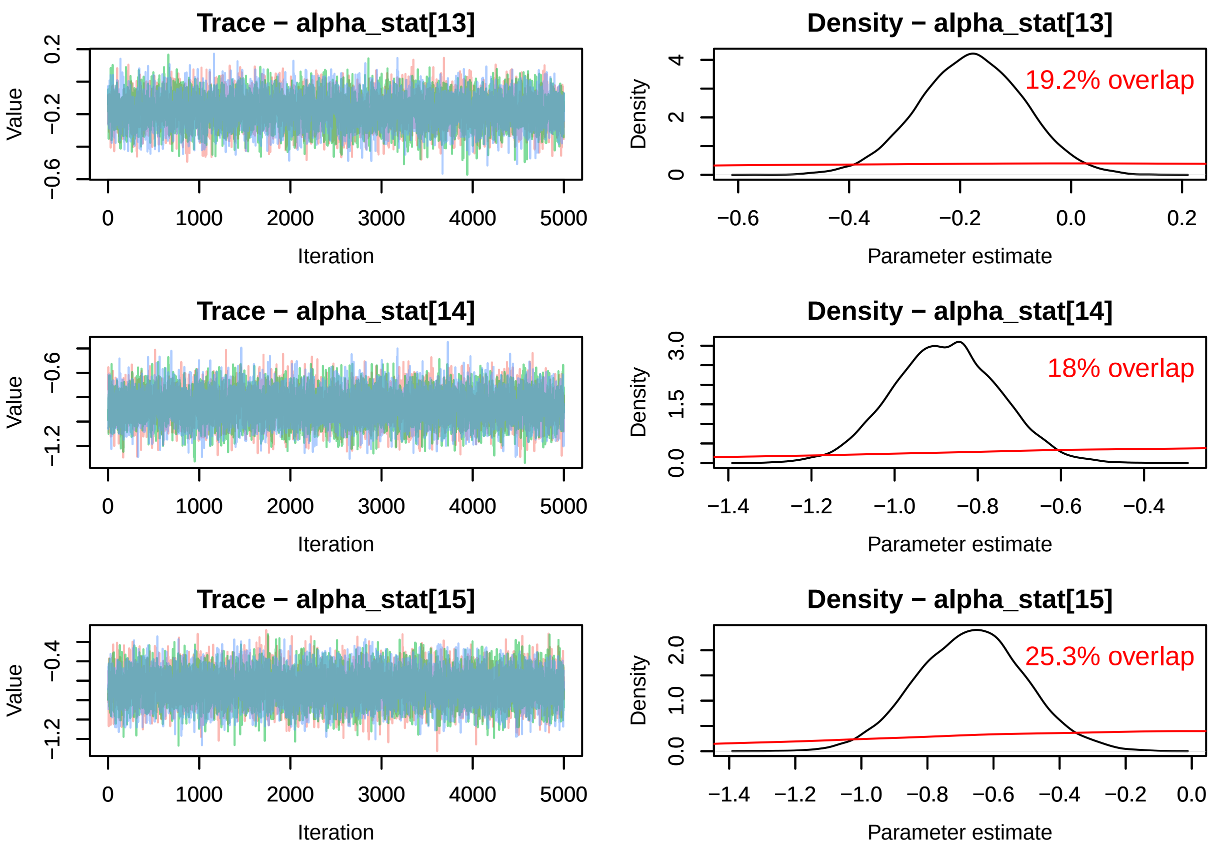
```
## Warning in MCMCtrace(chains, params = "alpha_stat", priors = PR, pdf = FALSE):
## Only one prior specified for > 1 parameter. Using a single prior for all
## parameters.
```

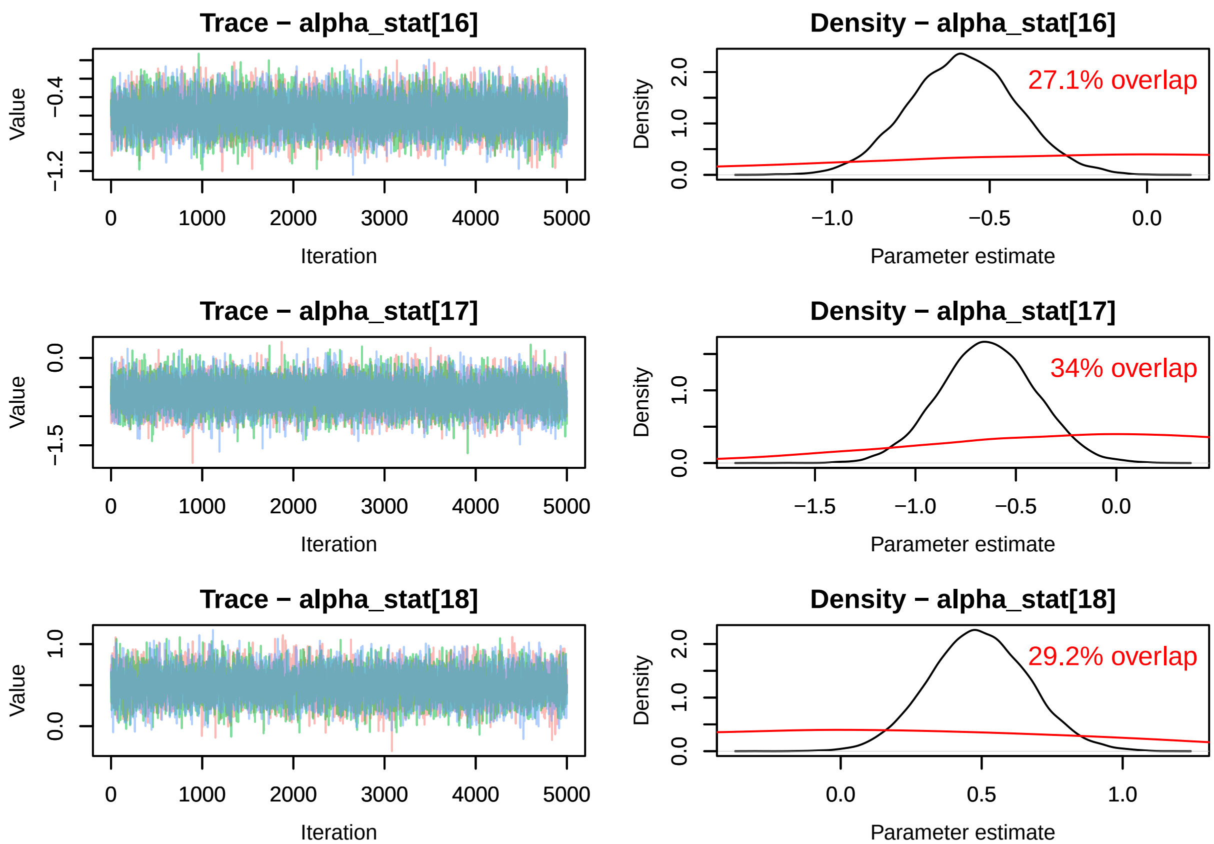

Trace − alpha_stat[1]
Density − alpha_stat[1]
12.8% overlap
Trace − alpha_stat[2]
Density − alpha_stat[2]
11.9% overlap
Trace − alpha_stat[3]
Density − alpha_stat[3]
11.5% overlap
Value
Density
Iteration
Parameter estimate


Trace − alpha_stat[4]
Value
Iteration
Density − alpha_stat[4]
23.2% overlap
Density
Parameter estimate
Trace − alpha_stat[5]
Value
Iteration
Density − alpha_stat[5]
22.9% overlap
Density
Parameter estimate
Trace − alpha_stat[6]
Value
Iteration
Density − alpha_stat[6]
35.1% overlap
Density
Parameter estimate


Trace − alpha_stat[7]
Value
Iteration
0 1000 2000 3000 4000 5000
Density − alpha_stat[7]
34.1% overlap
Density
Parameter estimate
−0.5 0.0 0.5 1.0 1.5
Trace − alpha_stat[8]
Value
Iteration
0 1000 2000 3000 4000 5000
Density − alpha_stat[8]
21.5% overlap
Density
Parameter estimate
0.4 0.6 0.8 1.0 1.2 1.4
Trace − alpha_stat[9]
Value
Iteration
0 1000 2000 3000 4000 5000
Density − alpha_stat[9]
29.4% overlap
Density
Parameter estimate
−1.0 −0.5 0.0


Trace − alpha_stat[10]
Value
−0.8
−0.2
0
1000
2000
3000
4000
5000
Iteration
Density − alpha_stat[10]
Density
0.0
1.5
3.0
23.1% overlap
−0.8
−0.6
−0.4
−0.2
0.0
0.2
Parameter estimate
Trace − alpha_stat[11]
Value
−0.6
0.0
0.6
0
1000
2000
3000
4000
5000
Iteration
Density − alpha_stat[11]
Density
0.0
1.0
2.0
31.8% overlap
−0.5
0.0
0.5
Parameter estimate
Trace − alpha_stat[12]
Value
−1.2
−0.6
0
1000
2000
3000
4000
5000
Iteration
Density − alpha_stat[12]
Density
0.0
1.5
3.0
18.7% overlap
−1.2
−1.0
−0.8
−0.6
−0.4
−0.2
Parameter estimate


Trace − alpha_stat[13]
Value
0.2
−0.2
−0.6
0
1000
2000
3000
4000
5000
Iteration
Density − alpha_stat[13]
Density
4
2
0
19.2% overlap
−0.6
−0.4
−0.2
0.0
0.2
Parameter estimate
Trace − alpha_stat[14]
Value
−0.6
−1.2
0
1000
2000
3000
4000
5000
Iteration
Density − alpha_stat[14]
Density
3.0
1.5
0.0
18% overlap
−1.4
−1.2
−1.0
−0.8
−0.6
−0.4
Parameter estimate
Trace − alpha_stat[15]
Value
−0.4
−1.2
0
1000
2000
3000
4000
5000
Iteration
Density − alpha_stat[15]
Density
2.0
1.0
0.0
25.3% overlap
−1.4
−1.2
−1.0
−0.8
−0.6
−0.4
−0.2
0.0
Parameter estimate


Trace − alpha_stat[16]
Value
Iteration
Density − alpha_stat[16]
Density
Parameter estimate
27.1% overlap
Trace − alpha_stat[17]
Value
Iteration
Density − alpha_stat[17]
Density
Parameter estimate
34% overlap
Trace − alpha_stat[18]
Value
Iteration
Density − alpha_stat[18]
Density
Parameter estimate
29.2% overlap